

SEMINAR

NÂNG CAO LẬP TRÌNH VỚI GENAI

NHÓM SINH VIÊN THỰC HIỆN

NGUYỄN HỮU ANH KHOA

NGUYỄN LÊ QUỲNH HƯƠNG

VÕ THÀNH DANH

TRANG GIA HUY

TỪ CƠ BẢN ĐẾN NÂNG CAO VỀ GỢI Ý LLM CHO LẬP TRÌNH GENAI

01

BEHIND THE SCENES: HOW CHATGPT, GITHUB
COPILOT, AND OTHER LLMS WORK

02

READING AND UNDERSTANDING CODE BASES
WITH GENAI

03

INTRODUCTION TO PROMPT ENGINEERING

04

ADVANCED PROMPT ENGINEERING FOR
CODING-RELATED TASKS

05

REFACTORING CODE WITH GENAI

06

FINE-TUNING MODELS WITH OPENAI

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

1. GIỚI THIỆU BỐI CẢNH

Các công cụ trí tuệ nhân tạo như Chat GPT hay GitHub Copilot đã thay đổi hoàn toàn cách chúng ta làm việc và sáng tạo

Để làm chủ công nghệ này, chúng ta cần hiểu được "cỗ máy" bên trong vận hành như thế nào thông qua:

- Nguyên lý hoạt động của LLM
- Kiến trúc tổng thể của LLM
- Cơ chế xử lý ngôn ngữ
- Hạn chế

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

2. NGUYÊN LÝ HOẠT ĐỘNG CỦA LLM

01

BẢN CHẤT CỦA LLM

LLM hoạt động hoàn toàn dựa trên xác suất thống kê

Để tạo ra một câu trả lời hoàn chỉnh, mô hình thực hiện một vòng lặp liên tục qua các bước sau

Để tạo ra một câu trả lời hoàn chỉnh, mô hình thực hiện một vòng lặp liên tục qua các bước sau:

- Tiếp nhận yêu cầu
- Phân tích ngữ cảnh
- Dự đoán quân cờ tiếp theo
- Vòng lặp không ngừng

02

TỪ MÔ HÌNH THỐNG KÊ ĐẾN AI

BÀI TOÁN VỀ SỰ BÙNG NỔ CỦA NGÔN NGỮ

Về mặt lý thuyết, ta có thể mô hình hóa ngôn ngữ bằng một công thức toán học đơn giản

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$

GIẢI PHÁP: NEURAL NETWORK

Sử dụng Neural network để "ước lượng" hoặc "xấp xỉ" hàm xác suất đó.

Neural network hoạt động như một bộ lọc thông minh:

- Học quy luật, không học vẹt
- Nén thông tin

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

3. KIẾN TRÚC TỔNG THỂ CỦA LLM

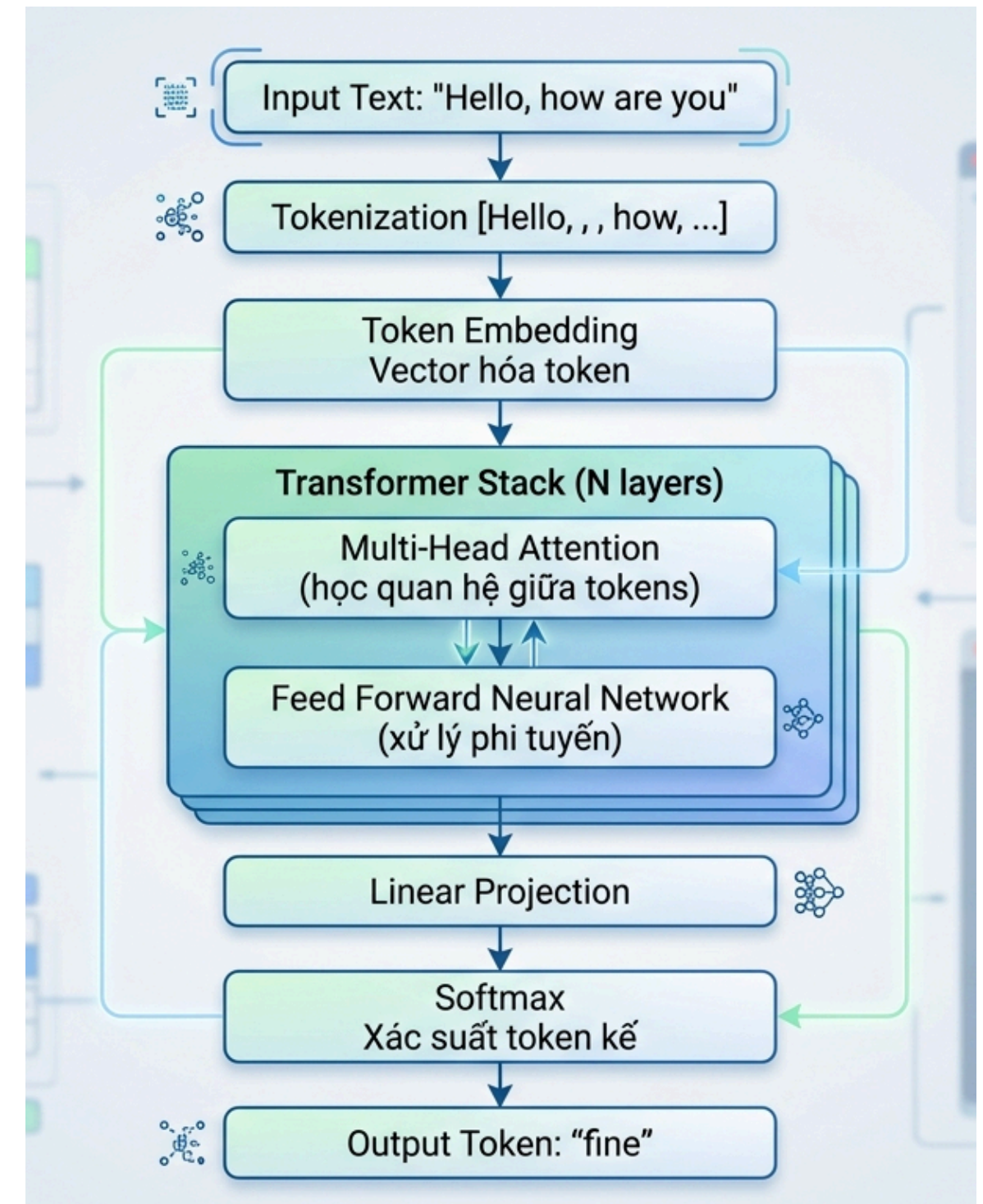
Ta có thể hình dung toàn bộ quá trình một LLM tạo ra câu chữ giống như một dây chuyền sản xuất hiện đại (pipeline).

Chia nhỏ dữ liệu (Tokenization)

Biến chữ thành số (Embedding)

Học mối quan hệ qua các lớp Transformer

Dự đoán và Hoàn thiện



BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

4. TOKENIZATION

TOKEN LÀ GÌ?

Token được định nghĩa là đơn vị dữ liệu nhỏ nhất mà hệ thống có thể tiếp nhận và xử lý.

CÁC PHƯƠNG PHÁP TOKENIZATION

Phân tách theo cấp độ ký tự (Character-level)

Phân tách theo cấp độ từ (Word-level)

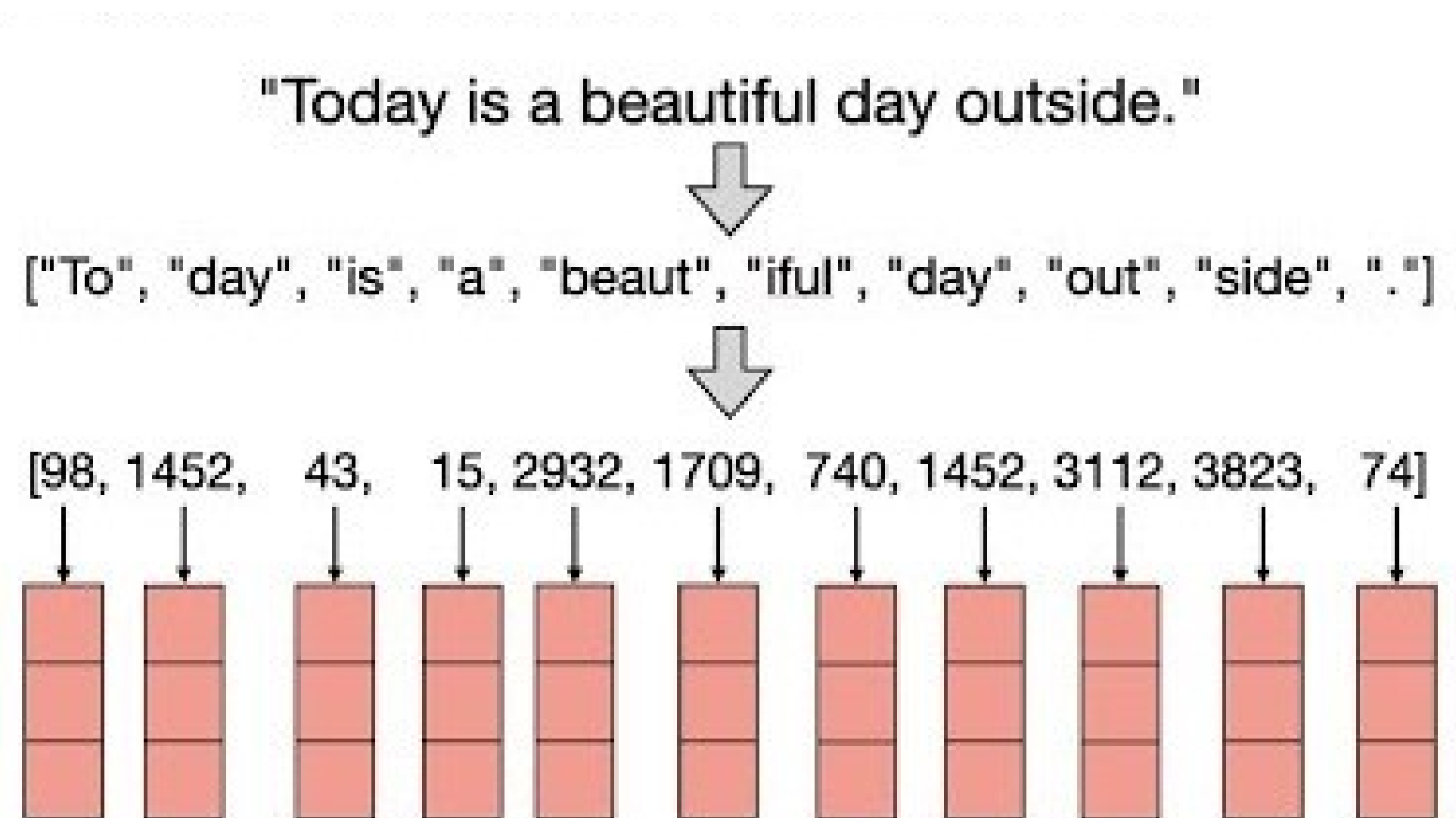
Phân tách theo cấp độ mảnh từ (Subword)

EMBEDDING

Trong giai đoạn này, mỗi token sẽ không còn là các ký tự chữ cái đơn thuần. Thay vào đó, nó được đại diện bởi một dãy số thực dài (được gọi là vector).

VAI TRÒ CỦA EMBEDDING TRONG VIỆC HIỂU NGÔN NGỮ

So sánh ngữ nghĩa, Hiểu mối quan hệ giữa các từ



BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

5. ATTENTION

CƠ CHẾ ATTENTION

Cơ chế Attention cho phép mô hình gán một "trọng số" cho từng token trong câu.

Công thức toán học cốt lõi

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

TẠI SAO CẦN NHIỀU "ĐẦU" CHÚ Ý?

Trong một câu văn, mối quan hệ giữa các từ rất đa tầng và phức tạp. Việc sử dụng nhiều "Head" cho phép mô hình học được các khía cạnh khác nhau của ngôn ngữ đồng thời

- Học về Ngữ pháp
- Học về Ngữ nghĩa
- Học về Vị trí

MULTI-HEAD ATTENTION

Thay vì chỉ sử dụng một bộ lọc "Attention" duy nhất, các mô hình ngôn ngữ hiện đại triển khai nhiều bộ lọc hoạt động song song

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

6. HẠN CHẾ

01

THIẾU SỰ HIỂU BIẾT THỰC SỰ VỀ THẾ GIỚI

01

HIỆN TƯỢNG "ẢO GIÁC" (HALLUCINATION)

01

SỰ PHỤ THUỘC VÀO DỮ LIỆU HUẤN LUYỆN

01

GIỚI HẠN VỀ "CỬA SỐ NGỮ CẢNH" (CONTEXT WINDOW)

7. KẾT LUẬN

Mọi câu trả lời thông minh mà chúng ta nhận được đều là kết quả của một dây chuyền xử lý nghiêm ngặt:

- Tokenization
- Embedding
- Attention
- Prediction

Việc nắm vững các thành phần kỹ thuật này không chỉ dành cho các chuyên gia, mà còn mang lại lợi ích thực tế cho mọi người dùng:

- Sử dụng AI hiệu quả hơn
- Thiết kế và nâng cấp hệ thống
- Tránh hiểu sai khả năng của AI

READING AND UNDERSTANDING CODE BASES WITH GENAI

1. GIỚI THIỆU

THỰC TRẠNG

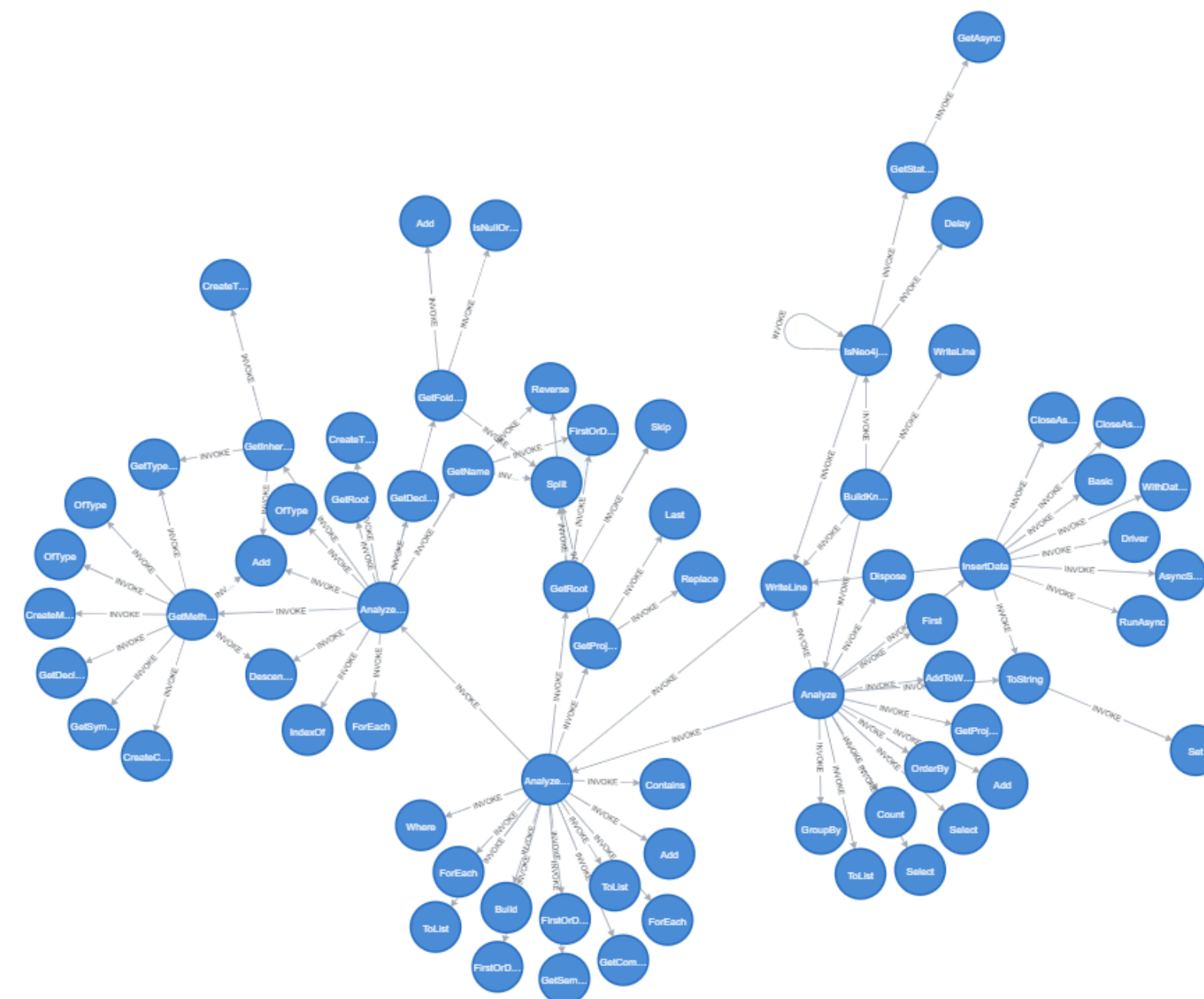
Codebase thực tế không chỉ là những dòng code đơn lẻ mà là một hệ sinh thái (Source code, Config, Docs, Docker).

KỸ NĂNG CỐT LÕI

Độc và hiểu nhanh một dự án lớn là "sống còn" đối với lập trình viên hiện đại.

GIẢI PHÁP

Sử dụng hệ sinh thái GenAI (GitHub Copilot, ChatGPT, OpenAI API) làm trợ lý phân tích



READING AND UNDERSTANDING CODE BASES WITH GENAI

2. CẤU TRÚC CODEBASE

Việc tuân thủ cấu trúc này mang lại ba lợi ích chiến lược cho công tác quản trị dự án:

- Xác định Entry Point: Biết luồng thực thi bắt đầu từ đâu.
- Minh bạch luồng dữ liệu: Tách biệt giữa giao diện (App) và xử lý (Logic).
- Quản lý Dependencies: Đảm bảo hệ thống nhất quán.

```
ch7/  
├─ app.py (Entry Point - Điểm bắt đầu)  
├─ src/ (Business Logic - Trái tim dự án)  
│   └─ manhattan.py  
├─ Dockerfile (Môi trường triển khai)  
├─ README.md (Tài liệu hướng dẫn)  
└─ requirements.txt (Danh sách thư viện)
```

SƠ ĐỒ CẤU TRÚC MẪU (PROJECT TREE)

READING AND UNDERSTANDING CODE

BASES WITH GENAI

3. BUSINESS LOGIC: MANHATTAN DISTANCE

Khoảng cách Manhattan (hay còn gọi là Taxicab geometry) là một phương pháp đo lường khoảng cách giữa hai điểm trong không gian n-chiều. Thay vì đo đường chéo trực tiếp như khoảng cách Euclidean, Manhattan Distance tính tổng độ lệch tuyệt đối giữa các tọa độ tương ứng của chúng.

Ứng dụng: Machine Learning, Xử lý hình ảnh (Image Processing)

CÔNG THỨC

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

CODE MINH HỌA

```
def get_manhattan_distance(  
    x1: Tuple[int, int],  
    x2: Tuple[int, int],  
) -> int:  
    x_dist: int = abs(x1[0] - x2[0])  
    y_dist: int = abs(x1[1] - x2[1])  
  
    dist: int = x_dist + y_dist  
    return dist
```


READING AND UNDERSTANDING CODE

BASES WITH GENAI

4. SỬ DỤNG GENAI ĐỂ HIỂU CODE

SỬ DỤNG GITHUB COPILOT ĐỂ GIẢI THÍCH CODE

Công cụ này cho phép tương tác trực tiếp với mã nguồn thông qua giao diện chat hoặc câu lệnh đặc biệt. Khi sử dụng lệnh /explain trên một đoạn mã cụ thể AI sẽ thực hiện:

- Giải mã chức năng:
- Làm rõ tham số và đầu ra
- Phân tích logic dòng lệnh

ĐÁNH GIÁ HIỆU QUẢ ỨNG DỤNG

Đặc điểm	Chi tiết
Ưu điểm	Tốc độ cực nhanh, tích hợp sẵn trong VS Code, giữ luồng tập trung.
Nhược điểm	Đôi khi giải thích quá tổng quát, chưa hiểu sâu quy tắc riêng của doanh nghiệp.

```
/explain whitespace before ':', blank line at end of file, Type "float" is not assignable to return type "float64"
"float" is not assignable to "float64", Object of type "Series[Any]" is not callable
Attribute "call" is unknown
```

manhattan.py:1-25

Here's how to address each issue in your code:

Issues & Fixes

1. Whitespace before ':'
Remove the space before the colon in type annotations, e.g. `coordinates_a : pd.DataFrame` → `coordinates_a: pd.DataFrame`.

2. Blank line at end of file
Ensure there is exactly one blank line at the end of the file.

3. Type "float" is not assignable to return type "float64"
The function `get_manhattan_distance` is annotated to return `np.float64`, but you are returning a Python `float`.
Fix: Convert the result to `np.float64` before returning.

4. Object of type "Series[Any]" is not callable
This usually means you tried to call a pandas Series as a function, e.g. `series()` instead of using a method or attribute.
In your code, this error does not appear directly, but double-check you are not calling a Series object.

READING AND UNDERSTANDING CODE

BASES WITH GENAI

4. SỬ DỤNG GENAI ĐỂ HIỂU CODE

TỐI ƯU HÓA PROMPT VỚI CHATGPT

Một prompt hiệu quả cho hàm tính khoảng cách Manhattan cần bao gồm 5 thành phần cốt lõi:

- Purpose (Mục đích): Mô tả rõ hàm dùng để làm gì
- Arguments (Đối số): Xác định kiểu dữ liệu đầu vào
- Data Flow (Luồng dữ liệu): Mô tả các bước xử lý
- Output (Đầu ra): Định dạng kết quả trả về
- Edge Cases (Trường hợp biên): Các tình huống lỗi cần xử lý.

Here is a structured explanation of the function based on your requested points:

1. Function's purpose

The function `get_manhattan_distance` computes the Manhattan distance (also known as L1 distance) between two datasets represented as Pandas DataFrames.

```
You, 3 days ago | 1 author (You)
1 CONTEXT: you are provided with:
2 1. A Python function enclosed with {{{ FUNCTION }}}
3 2. Explanation points enclosed with {{ POINTS }}. You, 3 days ago
4
5 TASK: Explain the function using the explanation points.
6
7 FUNCTION: {{{ def get_manhattan_distance(
8     df1: pd.DataFrame,
9     df2: pd.DataFrame,
10 ) -> float:
11     element_wise_dist: pd.DataFrame = (df1 - df2).abs()
12     dist: Union[np.int64, np.float64] = element_wise_dist.sum().sum()
13     return float(dist)
14 }}}
15
16 POINTS: {{{
17 1. Function's purpose
18 2. Arguments and their types
19 3. Step-by-step data flow
20 4. Output and its types
21 5. Potential edge cases
22 }}}
23
24 EXPLANATION:
```

READING AND UNDERSTANDING CODE BASES WITH GENAI

4. SỬ DỤNG GENAI ĐỂ HIỂU CODE

OPENAI API: TỰ ĐỘNG HÓA TRÊN QUY MÔ LỚN

Trong các dự án có quy mô lớn với hàng trăm module, việc sử dụng các công cụ tương tác thủ công như GitHub Copilot có thể gây tốn kém thời gian. Khi đó, việc tích hợp trực tiếp các mô hình ngôn ngữ (như gpt-4o-mini) qua API là giải pháp tối ưu.

NGŨ CẢNH ỨNG DỤNG (USE CASES)

- Cần tự động hóa (Automation): Tự động tạo tài liệu (Documentation) cho toàn bộ codebase mỗi khi có bản cập nhật mới.
- Phân tích quy mô lớn (Batch Processing): Quét và giải thích hàng loạt hàm (functions) cùng lúc để kiểm tra tính nhất quán của logic nghiệp vụ.

LỢI ÍCH CHIẾN LƯỢC

- Tính nhất quán: Đảm bảo mọi hàm trong hệ thống đều được giải thích theo một format duy nhất, giúp các kỹ sư mới (Onboarding) tiếp cận dự án nhanh hơn.
- Khả năng mở rộng: Có thể tích hợp trực tiếp vào luồng CI/CD (Continuous Integration/Continuous Deployment) để tự động kiểm tra logic hoặc cập nhật tệp README.md sau mỗi lần commit.

READING AND UNDERSTANDING CODE

BASES WITH GENAI

4. SỬ DỤNG GENAI ĐỂ HIỂU CODE

```
→ python -m ch7.prompts.openai_explain
```

Explanation: 1. **Function's purpose**: The function `get_manhattan_distance` is designed to calculate the Manhattan distance between two DataFrames. The Manhattan distance, also known as the L1 distance or taxicab distance, is the sum of the absolute differences of their corresponding elements.

2. **Arguments and their types**: The function takes two arguments:

- `df1`: A pandas DataFrame containing numerical data.
- `df2`: Another pandas DataFrame of the same shape and containing numerical data. The function assumes that both DataFrames have the same dimensions.

3. **Step-by-step data flow**:

- First, it computes the element-wise difference between the two DataFrames using `df1 - df2`.
- It then takes the absolute value of these differences with the `.abs()` method, resulting in a new DataFrame (`element_wise_dist`) where each element represents the absolute difference between the corresponding elements in `df1` and `df2`.
- Next, the function sums all the elements in the `element_wise_dist` DataFrame with `sum().sum()`. The first `sum()` computes the sum across columns, and the second `sum()` aggregates those sums into a total.
- Finally, this total sum is converted to a float type using `astype(float)` and returned.

4. **Output and its types**: The output of the function is a single value representing the Manhattan distance, and it is of type `np.float64`, which is a NumPy floating-point data type.

5. **Potential edge cases**: The function may encounter several edge cases, such as:

- If `df1` and `df2` do not have the same shape, it will raise a `ValueError` due to shape mismatch during the calculation of the difference.
- If either DataFrame is empty, the function will return a distance of `0.0`, which can be misleading as a valid output, but is mathematically correct in terms of the operation performed.
- If the DataFrames contain non-numeric data types, operations will fail, resulting in errors during subtraction and absolute value calculations.

READING AND UNDERSTANDING CODE

BASES WITH GENAI

5. HIỂU NON-PROJECT FILES

REQUIREMENTS.TXT

Trong một codebase chuyên nghiệp, việc định nghĩa rõ ràng các thư viện hỗ trợ là yêu cầu bắt buộc. Điều này được thực hiện thông qua tệp requirements.txt, giúp hệ thống xác định chính xác các phiên bản cần thiết để khởi chạy ứng dụng.

CẤU TRÚC VÀ PHIÊN BẢN (VERSIONING)

Việc chỉ định phiên bản cụ thể giúp tránh các lỗi tương thích khi thư viện cập nhật các hàm mới (Breaking changes).

- **pandas==2.2.3**: Thư viện cốt lõi dùng để xử lý cấu trúc dữ liệu DataFrame và thực hiện các phép toán trong hàm get_manhattan_distance.

DOCKERFILE

- FROM: Hệ điều hành nền.
- WORKDIR: Thư mục làm việc.
- COPY & RUN: Tận dụng Layer Caching để build nhanh.
- EXPOSE & CMD: Cổng giao tiếp và lệnh thực thi cuối cùng.

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

READING AND UNDERSTANDING CODE BASES WITH GENAI

5. HIỂU NON-PROJECT FILES

TẦM QUAN TRỌNG CỦA VIỆC XÁC ĐỊNH DEPENDENCIES

Trong một codebase chuyên nghiệp, việc định nghĩa rõ ràng các thư viện bổ trợ là yêu cầu bắt buộc. Điều này được thực hiện thông qua tệp requirements.txt, giúp hệ thống xác định chính xác các phiên bản cần thiết để khởi chạy ứng dụng.

Việc quản lý chặt chẽ các thư viện mang lại 3 lợi ích cốt lõi:

- Tính đóng gói (Portability): Khi kết hợp với Dockerfile, hệ thống sẽ tự động cài đặt đúng các phiên bản này, đảm bảo môi trường phát triển (Local) và môi trường thực tế (Production) hoàn toàn đồng nhất.
- Xác định phạm vi ứng dụng: Nhìn vào requirements.txt, một lập trình viên có thể hiểu ngay công nghệ chủ đạo của dự án (Ví dụ: Dự án này là một Web Service xử lý dữ liệu).
- Bảo mật và Bảo trì: Dễ dàng kiểm tra xem các phiên bản đang dùng có lỗ hổng bảo mật nào không và lên kế hoạch nâng cấp định kỳ.

READING AND UNDERSTANDING CODE BASES WITH GENAI

6. DEBUGGING – CÁCH HIỂU CODE HIỆU QUẢ NHẤT

Việc đọc mã nguồn thuần túy thường khó giúp chúng ta hình dung được sự thay đổi của dữ liệu qua từng bước. Do đó, quy trình thực nghiệm dưới đây được khuyến nghị áp dụng để nắm bắt logic hệ thống một cách triệt để.

QUY TRÌNH THỰC NGHIỆM 3 BƯỚC

- Khởi tạo dữ liệu mẫu (Mock Data): Tạo các bảng dữ liệu nhỏ (DataFrame) với các giá trị đã biết trước để dễ dàng đối chiếu kết quả.
- Chạy Debug (Debugging): Sử dụng các công cụ debug trong IDE để dừng chương trình tại từng dòng lệnh.
- Quan sát biến (Variable Inspection): Theo dõi giá trị của các biến trong bộ nhớ tại thời điểm thực thi.

READING AND UNDERSTANDING CODE

BASES WITH GENAI

7. TẠO DỮ LIỆU MẪU BẰNG AI

Việc chạy Debug và quan sát biến chỉ thực sự hiệu quả khi chúng ta có một tập dữ liệu mẫu (Mock Data) đủ tốt. Sự kết hợp giữa GitHub Copilot và ChatGPT giúp lập trình viên nhanh chóng tạo ra các bộ dữ liệu mô phỏng sát với thực tế.

SỬ DỤNG GITHUB COPILOT ĐỂ TỰ SINH MÃ NGUỒN (AUTO-GENERATION)

GitHub Copilot thể hiện thế mạnh trong việc dự đoán ngữ cảnh. Khi bạn bắt đầu viết hàm kiểm thử, Copilot có thể:

- Tự sinh cấu trúc DataFrame: Chỉ cần gõ tên biến `df_test`, AI sẽ gợi ý các dòng mã tạo dữ liệu mẫu với thư viện Pandas.
- Đề xuất giá trị ngẫu nhiên: Sử dụng `numpy.random` để tạo ra các ma trận số thực phù hợp với đầu vào của hàm `Manhattan`.

TÙY CHỈNH DỮ LIỆU CHUYÊN SÂU VỚI CHATGPT

- Điều chỉnh quy mô: Yêu cầu tạo dữ liệu với số dòng và số cột cực lớn để kiểm tra hiệu năng tính toán.
- Kiểm soát kiểu dữ liệu: Yêu cầu xen kẽ các cột dữ liệu số (Integer, Float) với dữ liệu chuỗi (String) để kiểm thử khả năng xử lý lỗi của hệ thống.
- Mô phỏng dữ liệu nhiễu: Tạo các DataFrame chứa giá trị rỗng (NaN) hoặc giá trị ngoại lai (Outliers) để quan sát cách hàm `get_manhattan_distance` phản ứng trong các trường hợp biên.

READING AND UNDERSTANDING CODE

BASES WITH GENAI

8. SO SÁNH CÔNG CỤ

Công cụ	Ưu điểm	Nhược điểm
Copilot	Nhanh	Ít tùy chỉnh
ChatGPT	Linh hoạt	Cần viết prompt
OpenAI API	Tự động hóa	Phức tạp

LỢI ÍCH CỦA VIỆC KẾT HỢP AI TRONG TESTING

Sự phối hợp này tạo ra một quy trình kiểm thử khép kín:

- ChatGPT thiết kế kịch bản dữ liệu phức tạp.
- Copilot thực thi mã nguồn khởi tạo nhanh chóng trên IDE.
- Lập trình viên thực hiện Debug để quan sát giá trị trung gian.

9. KẾT LUẬN

- 01
- THAY ĐỔI TƯ DUY TIẾP CẬN MÃ NGUỒN
- 02
- VAI TRÒ CHIẾN LƯỢC CỦA GENERATIVE AI (GENAI)
- 03
- DEBUGGING: KỸ NĂNG ĐỊNH HÌNH NĂNG LỰC KỸ SƯ
- 04
- MÔ HÌNH PHÂN TÍCH HIỆN ĐẠI

INTRODUCTION TO PROMPT ENGINEERING

1. GIỚI THIỆU

PROMPT ENGINEERING LÀ GÌ?

Prompt Engineering – một lĩnh vực quan trọng trong việc làm việc với các mô hình ngôn ngữ lớn (LLMs) như ChatGPT, GitHub Copilot và OpenAI API.

Prompt Engineering giúp lập trình viên:

- kiểm soát output của AI
- tạo ra code chính xác hơn
- đảm bảo tính nhất quán trong coding style



INTRODUCTION TO PROMPT ENGINEERING

2. FEW-SHOT LEARNING

FEW-SHOT LEARNING LÀ GÌ?

Few-shot learning (hay còn gọi là in-context learning) là kỹ thuật cung cấp cho mô hình một số ví dụ đầu vào và đầu ra để hướng dẫn cách xử lý dữ liệu mới.

Thay vì chỉ mô tả yêu cầu bằng ngôn ngữ tự nhiên, ta đưa trực tiếp các ví dụ:

INPUT_1 → OUTPUT_1

INPUT_2 → OUTPUT_2

INPUT → ?

ỨNG DỤNG FEW-SHOT LEARNING TRONG LẬP TRÌNH

Trong lĩnh vực lập trình, kỹ thuật này được ứng dụng rộng rãi nhằm kiểm soát cách sinh mã nguồn của mô hình, đảm bảo tính nhất quán và phù hợp với yêu cầu cụ thể của hệ thống:

- Chuẩn hóa coding style
- Chuẩn hóa logging và comment
- Đánh giá

INTRODUCTION TO PROMPT ENGINEERING

2. FEW-SHOT LEARNING

CHUẨN HÓA CODING STYLE

```
style_guide.py M ×
ch8 > code_samples > style_guide.py > ...
1  import numpy as np
2
3  def get_area(
4      radius: float,
5      ) -> float:
6      area: float = np.pi * radius ** 2
7      return area
8
9
10 def get_arithmetic_mean(
11     x1: float,
12     x2: float,
13     ) -> float:
14     arithmetic_mean: float = (x1 + x2) / 2
15     return arithmetic_mean
16
```

MÃ NGUỒN BAN ĐẦU

```
style_guide.py M  distances.py 1, U ●
ch8 > code_samples > distances.py > get_eucleadian_distance
1  def get_eucleadian_distance(
        x1: float,
        x2: float,
        y1: float,
        y2: float,
        ) -> float:
    distance: float = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
    return distance
```

MÃ NGUỒN ĐÃ ĐƯỢC ĐỊNH DẠNG

INTRODUCTION TO PROMPT ENGINEERING

2. FEW-SHOT LEARNING

CHUẨN HÓA LOGGING VÀ COMMENT

```
CONTEXT: You are provided with:
1. Python snippet enclosed with {{{ OLD }}}
2. Examples enclosed with {{{ OLD_i }}} followed by a desirable output
enclosed with {{{ REFACTORED_i }}}

TASK: Refactor the snippet to a log message.

OLD_1: {{{ print('Process started for config.txt with verbose=True') }}}
REFACTORED_1: {{{ logger.info('Processing started', extra={'verbose':
True, file: 'config.txt'}) }}}

OLD_2: {{{ print('Warning! Could not load user U-232 data from user_info.
csv') }}}
REFACTORED_2: {{{ logger.warning('User data failed to load',
extra={'user': 'U-232', 'file': 'user_info.csv'}) }}}

OLD: {{{ print('Error! File not found: passwords.txt') }}}
REFACTORED: Hila Paz Herszfang, 9 months ago • ch8
```

ĐÁNH GIÁ

Few-shot learning mang lại nhiều lợi ích trong lập trình:

- không yêu cầu huấn luyện lại mô hình
- dễ triển khai và linh hoạt
- cải thiện đáng kể chất lượng mã nguồn sinh ra

Tuy nhiên, hiệu quả của phương pháp phụ thuộc vào chất lượng và độ đại diện của các ví dụ được cung cấp.

INTRODUCTION TO PROMPT ENGINEERING

3. ITERATIVE PROMPTING

ITERATIVE PROMPTING LÀ GÌ?

Iterative prompting là kỹ thuật tương tác lặp lại với mô hình trong một vòng phản hồi (feedback loop), trong đó đầu ra sẽ được cải thiện dần dựa trên việc đánh giá các phản hồi trước đó.

Phản hồi này có thể:

- Thủ công: ví dụ yêu cầu ChatGPT sửa lại câu trả lời để khắc phục lỗi chưa xác định
- Tự động: ví dụ compile code do API tạo ra hoặc chạy qua unit test

```
for i in range(100):
    try:
        exec(user_code)
        print(f"successfully compiled:\n {user_code}")
        break

    except SyntaxError as se:
        user_code = get_refined_output(user_code, traceback.format_exc())
```


INTRODUCTION TO PROMPT ENGINEERING

4. TEMPLATE-BASED PROMPTING

TEMPLATE-BASED PROMPTING LÀ GÌ?

Template-based prompting là kỹ thuật định nghĩa trước cấu trúc output, thay vì chỉ đưa ví dụ.

CONTEXT: You will be provided with Python code enclosed with `{{{ CODE }}}` that does not compile, and an error traceback enclosed with `{{{ TRACEBACK }}}`, along with a template of the output enclosed with `{{{ TEMPLATE }}}.`

TASK: Fix the errors in the code using the provided template.

TEMPLATE: This code requires a fix because of `{ ERROR TYPE }`. The fixed code is `{ CODE }`.

LỢI ÍCH

- dễ tái sử dụng
- chuẩn hóa prompt
- kiểm soát output chặt chẽ

INTRODUCTION TO PROMPT ENGINEERING

5. CHAIN-OF-THOUGHT PROMPTING (COT)

TEMPLATE-BASED PROMPTING LÀ GÌ?

Chain-of-Thought (CoT) prompting là kỹ thuật hướng dẫn mô hình suy luận theo từng bước (step-by-step) để đi đến kết quả cuối cùng. Thay vì yêu cầu mô hình đưa ra đáp án ngay lập tức, phương pháp này giúp mô hình phân tách bài toán thành các bước nhỏ, từ đó cải thiện độ chính xác và tính logic của kết quả.

```
def validate_no_bugs_in_source_code(sc: str) -> None:
    validate_syntax(sc)
    validate_compilation(sc)
    validate_reproducibility_across_runs(sc)
```

CÁCH THIẾT KẾ NÀY CHO PHÉP:

- Tách biệt rõ ràng giữa logic tổng thể và chi tiết triển khai
- Tận dụng khả năng của GenAI trong việc tự động sinh phần cài đặt cấp thấp

ADVANCED PROMPT ENGINEERING FOR CODING-RELATED TASKS

1. GIỚI THIỆU

Tập trung vào việc giải quyết các bài toán lập trình thực tế có độ phức tạp cao. Trọng tâm của chương là quy trình:

Đánh giá Baseline → Cải thiện bằng CoT → Tối ưu bằng Prompt Chaining

Ví dụ xuyên suốt trong chương là bài toán tính geometric mean, được dùng để so sánh hiệu quả của các phương pháp prompting khác nhau.

2. MỞ RỘNG PHẠM VI THỰC THI CỦA PROMPT (EXTENDING PROMPT SCOPE)

Trong thực tế, các bài toán lập trình thường yêu cầu xử lý logic đa bước thay vì chỉ một công thức toán học duy nhất

Một lỗi phổ biến của các Prompt cơ bản là không xử lý được các trường hợp biên hoặc dữ liệu không hợp lệ

Ví dụ:

Vấn đề: Các Prompt cơ bản thường thất bại khi gặp số âm trong tính toán Geometric Mean.

Giải pháp: Cung cấp ngữ cảnh nghiệp vụ (Business Logic). Chuyển đổi Net Returns sang Gross Returns ($\$1 + r\$$) trước khi tính toán để đảm bảo dữ liệu luôn dương.

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

3. BASELINE PROMPT VÀ KẾT QUẢ BAN ĐẦU

GITHUB COPILOT BASELINE

```
def get_geometric_mean(  
    net_returns: Dict[str, float],  
) -> float:  
    product: float = 1  
    for key in net_returns:  
        product *= net_returns[key]  
    geometric_mean: float = product ** (1 / len(net_returns))  
    return geometric_mean
```

ADVANCED PROMPT ENGINEERING FOR CODING-RELATED TASKS

3. BASELINE PROMPT VÀ KẾT QUẢ BAN ĐẦU

GITHUB COPILOT BASELINE

```
CONTEXT: You are provided with a Python function signature enclosed with {{{ FUNCTION }}}.  
TASK: implement the function.
```

```
FUNCTION: {{{ def get_geometric_mean(net_returns: Dict[str, float]) -> float: }}}}
```

```
CODE: Hila Paz Herszfang, 9 months ago • ch9 ...
```

```
def get_geometric_mean(net_returns: Dict[str, float]) -> float:  
    """ [... omitted for brevity ...] """  
    if not net_returns:  
        return 0.0  
    # Calculate the product of (1 + return) for each net return  
    product = math.prod(1 + r for r in net_returns.values())  
    n = len(net_returns)  
    # Compute the nth root of the product and subtract 1 to get the  
    geometric mean  
    geometric_mean = product ** (1 / n) - 1  
    return geometric_mean
```

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

3. BASELINE PROMPT VÀ KẾT QUẢ BAN ĐẦU

OPENAI API BASELINE

```
def get_geometric_mean(net_returns: Dict[str, float]) -> float:
    # Return 0.0 if there are no returns (could alternatively raise an
    # exception)
    if not net_returns:
        return 0.0
    import math

    # Calculate the product of (1 + return) for each period
    product = 1.0
    n = len(net_returns)
    for r in net_returns.values():
        product *= (1 + r)
```

```
SURROUND = "You are provided with a Python function signature enclosed with {{{ FUNCTION }}}."
SINGLE_TASK = "Your task is to implement the function."

SRC_CODE = """def get_geometric_mean(\n\t net_returns: Dict[str, float],\n) -> float: """

def get_user_prompt(src: str) -> str:
    return f"""
    FUNCTION: {{{{src}}}}

    CODE:
    """
```

```
# Compute the geometric mean: nth root of product then subtract 1
geom_mean = math.pow(product, 1 / n) - 1
return geom_mean
```

ADVANCED PROMPT ENGINEERING FOR CODING-RELATED TASKS

4. KỸ THUẬT CHAIN-OF-THOUGHT (COT) TRONG LẬP TRÌNH

CHAIN-OF-THOUGHT LÀ GÌ?

Chain-of-Thought (Chuỗi suy nghĩ) là kỹ thuật hướng dẫn mô hình thực hiện các bước lập luận trung gian trước khi đưa ra câu trả lời cuối cùng

Thay vì yêu cầu LLM "viết mã ngay lập tức", lập trình viên yêu cầu mô hình "giải thích logic từng bước".

=> Điều này đặc biệt hiệu quả với các mô hình nhỏ hơn như GPT-4o-mini, giúp chúng đạt được hiệu suất tương đương các mô hình lớn hơn khi xử lý logic phức tạp.

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

5. SỬ DỤNG COT (CHAIN OF THOUGHT) VỚI GITHUB COPILOT

Ý TƯỞNG CHÍNH

CoT (Chain of Thought) là kỹ thuật chia nhỏ bài toán thành các bước logic rõ ràng, giúp Copilot sinh code dễ hiểu, sạch hơn và chính xác hơn.

Thay vì yêu cầu viết một hàm phức tạp ngay lập tức, ta hướng dẫn Copilot theo từng bước.

Đối với bài toán tính trung bình hình học (geometric mean) từ tập net returns, ta có thể chia thành ba bước:

- Chuyển đổi từ net returns sang gross returns
- Tính geometric mean của gross returns
- Chuyển đổi kết quả từ gross returns về net returns

```
def get_average_return(  
    net_returns: Dict[str, float],  
) -> float:  
    gross_returns: np.ndarray = get_gross_returns(net_returns)  
    gross_average: float = get_geometric_mean(gross_returns)  
    net_average: float = get_net_average(gross_average)  
    return net_average
```

```
def get_gross_returns(  
    net_returns: Dict[str, float],  
) -> np.ndarray:  
    gross_returns: np.ndarray = np.array(list(net_returns.values())) + 1  
    return gross_returns  
  
def get_geometric_mean(  
    gross_returns: np.ndarray,  
) -> float:  
    gross_average: float = np.prod(gross_returns) ** (1 / len(gross_returns))  
    return gross_average  
  
def get_net_average(  
    gross_average: float,  
) -> float:  
    net_average: float = gross_average - 1  
    return net_average
```

Hila Paz Herszfang, 9 months ago • ch9

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

6. ÁP DỤNG CHAIN OF THOUGHT (COT) VỚI CHATGPT VÀ OPENAI API

COT VỚI CHATGPT

Tương tự như GitHub Copilot, kỹ thuật Chain of Thought (CoT) cũng có thể được áp dụng hiệu quả với ChatGPT trong các bài toán lập trình. Thay vì chỉ cung cấp chữ ký hàm, ta cung cấp toàn bộ hàm tổng (high-level function) và yêu cầu model hoàn thiện các hàm con còn thiếu.

```
CONTEXT: You are provided with a Python function enclosed with {{{ FUNCTION }}} that calls functions that should be completed.
TASK: Implement the missing functions.
FUNCTION: {{{def get_average_return(
    net_returns: Dict[str, float],
) -> float:
    gross_returns: np.ndarray = get_gross_returns(net_returns)
    gross_average: float = get_geometric_mean(gross_returns)
    net_average: float = get_net_average(gross_average)
    return net_average }}}
CODE: Hila Paz Herszfang, 9 months ago • ch9 ...
```


ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

6. ỨNG DỤNG CHAIN OF THOUGHT (COT) VỚI CHATGPT VÀ OPENAI API

COT VỚI CHATGPT

```
def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """ ... omitted for brevity ... """
    return np.array([1 + net_return for net_return in net_returns.
values()])

def get_geometric_mean(values: np.ndarray) -> float:
    """ ... omitted for brevity ... """
    return np.prod(values) ** (1 / len(values))

def get_net_average(gross_average: float) -> float:
    """ ... omitted for brevity ... """
    return gross_average - 1

def get_average_return(net_returns: Dict[str, float]) -> float:
    gross_returns: np.ndarray = get_gross_returns(net_returns)
    gross_average: float = get_geometric_mean(gross_returns)
```

```
net_average: float = get_net_average(gross_average)
return net_average
```

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

6. ÁP DỤNG CHAIN OF THOUGHT (COT) VỚI CHATGPT VÀ OPENAI API

COT VỚI OPENAI API

Kỹ thuật CoT cũng có thể áp dụng tương tự khi sử dụng OpenAI API. Ta cung cấp hàm tổng và yêu cầu model hoàn thiện các hàm còn thiếu thông qua system prompt và user prompt.

```
SURROUND = "You are provided with a Python function enclosed with {{{ FUNCTION }}} that calls functions that should be completed."
SINGLE_TASK = "Your task is to implement the missing functions."

def get_user_prompt(func: callable) -> str:
    return f"""
FUNCTION: {{{{ inspect.getsource(func) }}}}
CODE:
"""
```

```
def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """Convert net returns to gross returns."""
    return np.array([1 + r for r in net_returns.values()])

def get_geometric_mean(gross_returns: np.ndarray) -> float:
    """Calculate the geometric mean of a numpy array of gross returns."""
    return np.prod(gross_returns) ** (1 / len(gross_returns))

def get_net_average(gross_average: float) -> float:
    """Convert gross average back to net average."""
    return gross_average - 1

def get_average_return(net_returns: Dict[str, float]) -> float:
    ... omitted for brevity ...
```

ADVANCED PROMPT ENGINEERING FOR CODING-RELATED TASKS

7. SỬ DỤNG PROMPT CHAINING VỚI CÁC MÔ HÌNH LLM

GIỚI THIỆU

Prompt Chaining là một kỹ thuật quan trọng trong prompt engineering, cho phép cải thiện chất lượng đầu ra của các mô hình ngôn ngữ lớn (LLMs) bằng cách chia nhỏ bài toán thành nhiều bước xử lý tuần tự.

ÁP DỤNG PROMPT CHAINING VỚI CHATGPT

Khi áp dụng Prompt Chaining với ChatGPT, có hai nguyên tắc quan trọng:

- Cùng ngữ cảnh:
 - Chỉ cần đưa ra yêu cầu mới (không cần lặp lại toàn bộ prompt)
- Khác ngữ cảnh (ví dụ: viết test):
 - Cần viết lại prompt đầy đủ theo các best practices (như quy tắc five S's)

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

7. SỬ DỤNG PROMPT CHAINING VỚI CÁC MÔ HÌNH LLM

Đối với bài toán `get_average_return`, quá trình chaining có thể thực hiện theo các bước:

- Prompt 1: Sinh code hoàn chỉnh sử dụng CoT
- Prompt 2: Thêm type hints cho tất cả biến
- Prompt 3: Chuẩn hóa docstring theo Google Style
- Prompt 4: Viết unit test (tạo ngữ cảnh mới)

VÍ DỤ PROMP 3:

```
def get_geometric_mean(gross_returns: np.ndarray) -> float:
    """Calculates the geometric mean of gross returns.
    Args:
        gross_returns (np.ndarray): An array of gross returns.

    Returns:
        float: The geometric mean of the gross returns.
    """
    ... omitted for brevity ...

def get_net_average(gross_average: float) -> float: """Converts gross
average return to net average return.
Args:
    gross_average (float): The geometric mean of gross returns.
```

```
Returns:
    float: The corresponding net average return.
    """
    ... omitted for brevity ...

def get_average_return(net_returns: Dict[str, float]) -> float:
    """Calculates the geometric mean of net returns from a dictionary of net
    returns.
    Args:
        net_returns (Dict[str, float]): A dictionary of net returns per
        period.

    Returns:
        float: The geometric mean of the net returns.
    """
    ... omitted for brevity ...
```

ADVANCED PROMPT ENGINEERING

FOR CODING-RELATED TASKS

7. SỬ DỤNG PROMPT CHAINING VỚI CÁC MÔ HÌNH LLM

TỐI ƯU PROMPT CHAINING VỚI SELECTIVE HISTORY TRONG OPENAI API

Khi dùng API, lịch sử hội thoại dài sẽ làm tăng chi phí và gây nhiễu.

- Kỹ thuật: Chỉ lấy phần Code từ phản hồi trước, kết hợp với Prompt mới.
- Lợi ích: Giảm lượng Token tiêu thụ tới 75%, tăng tốc độ phản hồi và độ tập trung của Model.

GIẢI PHÁP SELECTIVE HISTORY

```
def get_refactor_user_prompt(assistant_output: str) -> str:
    code: str = re.sub(
        r"(.*)```python(.*)```(.*)",
        r"\2",
        assistant_output,
        flags=re.DOTALL
    ).strip()

    user_prompt: str = f"""
    OLD: {{{{{{ {code} }}}}}}
    REFACTORED:
    """
    return user_prompt
```

```
prompt_1: str = f"{SURROUND} {SINGLE_TASK}"
prompt_2: str = "You are provided with a Python code enclosed in {{{
FUNCTION }}}. Your task is to add type hints to all variables."
prompt_3: str = "You are provided with a Python code enclosed in {{{
FUNCTION }}}. Your task is to include Google Style docstring."
```

```
for prompt, func in prompts.items():
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": func(next_input)},
        ],
    )
    next_input = completion.choices[0].message.content
```


ADVANCED PROMPT ENGINEERING FOR CODING-RELATED TASKS

7. SỬ DỤNG PROMPT CHAINING VỚI CÁC MÔ HÌNH LLM

ÁP DỤNG PROMPT CHAINING VỚI GITHUB COPILOT

PROMPT THÊM DOCSTRING

Trong VS Code hoặc PyCharm, sử dụng Copilot Chat với prompt:

Add Google Style docstring to all functions.

Kết quả Copilot sẽ:

- Tự động thêm docstring cho tất cả các hàm
- Sử dụng đúng chuẩn Google Style
- Mô tả chính xác chức năng của từng hàm

```
def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """Converts net returns to gross returns.

    Args:
        net_returns (Dict[str, float]): A dictionary of net returns.

    Returns:
        np.ndarray: Array of gross returns.
    """
```

REFACTORING CODE WITH GENAI

1. GIỚI THIỆU

Refactoring là quá trình chỉnh sửa, tinh chỉnh hệ thống mã nguồn hiện tại nhằm cải thiện cấu trúc bên trong mà không làm thay đổi hành vi và chức năng vốn có của chương trình

2. VAI TRÒ CỦA REFACTORING

- Cải thiện khả năng đọc (Readability)
- Tối ưu hóa cấu trúc (Structure)
- Nâng cao hiệu năng (Performance)

3. VÍ DỤ MINH HỌA: HÀM TÍNH KHOẢNG CÁCH

Phân tích phiên bản mã nguồn ban đầu

Nhận diện "Code Smell" (Dấu hiệu mã nguồn xấu)

Định hướng Refactoring

REFACTORING CODE WITH GENAI

4. CÁC LOẠI REFACTORING

Cải thiện khả năng đọc (Readability)

Thay đổi cấu trúc (Structure)

Tối ưu hiệu năng (Performance)

5. HẠN CHẾ CỦA CÔNG CỤ IDE TRUYỀN THỐNG

Khả năng của các IDE phổ biến (VS Code, PyCharm, v.v.)

Những hạn chế của công cụ truyền thống

Sự trỗi dậy của GenAI trong Refactoring

6. ỨNG DỤNG GENAI TRONG REFACTORING

- Chain-of-Thought (CoT): Hướng dẫn AI suy luận qua các giai đoạn: Parse (Phân tích) → Classify (Phân loại) → Execute (Thực thi).
- Interactive Refactoring: Cung cấp cả đoạn Code cũ (Reference) để AI hiểu hành vi hiện tại và mẫu Cấu trúc mới (Target Structure) mà bạn muốn hướng tới
- Automation (API): Tích hợp vào pipeline CI/CD để tự động đề xuất bản vá cho các đoạn mã chưa tối ưu.

REFACTORING CODE WITH GENAI

7. REFACTORING HIỆU NĂNG VỚI GENAI

CHUYỂN ĐỔI TƯ DUY: TỪ SCALAR SANG VECTOR

```
import math

for i in range(n):
    for j in range(m):
        dist += (a[i][j] - b[i][j]) ** 2
result = math.sqrt(dist)
```

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
result = np.sqrt(np.sum((a - b) ** 2))
```

REFACTORING CODE WITH GENAI

7. REFACTORING HIỆU NĂNG VỚI GENAI

LỢI ÍCH CỦA VIỆC REFACTOR HIỆU NĂNG QUA GENAI

- 01 GIẢM THỜI GIAN THỰC THI (RUNTIME)
- 02 TẬN DỤNG TỐI ƯU PHẦN CỨNG
- 03 MÃ NGUỒN NGẮN GỌN (CLEAN CODE)

VAI TRÒ CỦA GENAI TRONG QUY TRÌNH NÀY

- 01 NHẬN DIỆN CƠ HỘI
- 02 ĐỀ XUẤT THƯ VIỆN PHÙ HỢP
- 03 KIỂM CHỨNG

REFACTORING CODE WITH GENAI

8. SO SÁNH CÁC CÔNG CỤ GENAI

Công cụ	Ưu điểm
GitHub Copilot	Nhanh, tích hợp trực tiếp IDE
ChatGPT	Hiểu logic tốt, dễ dùng
OpenAI API	Tự động hóa quy mô lớn

FINE-TUNING MODELS WITH OPENAI

1. GIỚI THIỆU

Fine-tuning (Tinh chỉnh) là quá trình can thiệp sâu hơn vào "tư duy" của mô hình để đạt được độ chính xác tuyệt đối cho các tác vụ chuyên biệt.

PROMPT ENGINEERING VS. FINE-TUNING

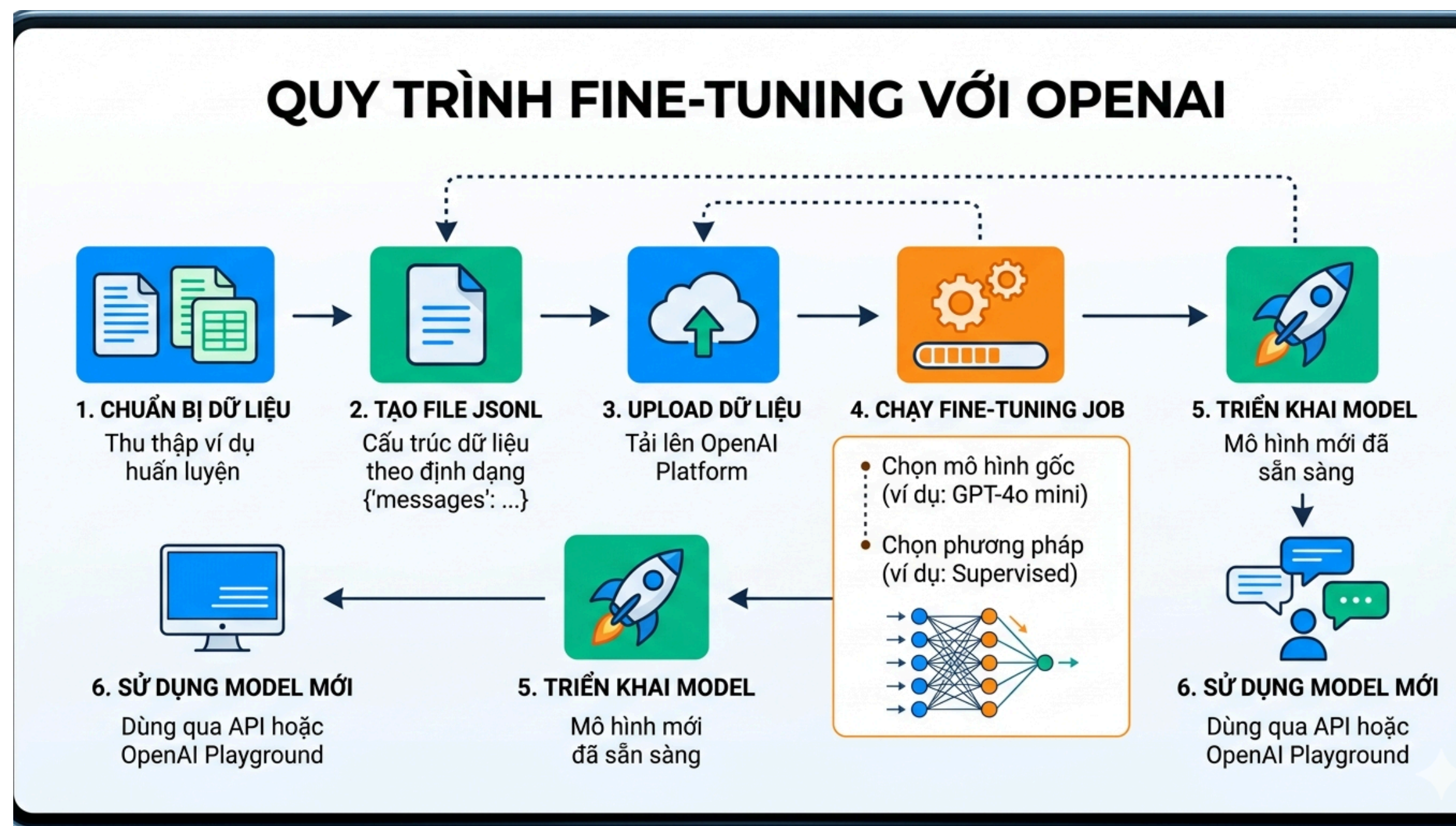
Đặc điểm	Prompt Engineering	Fine-tuning
Chi phí thiết lập	Thấp (chỉ cần viết prompt)	Cao (cần dữ liệu và GPU)
Độ chính xác	Phụ thuộc vào ngữ cảnh	Rất cao và ổn định
Khả năng cập nhật	Tức thời	Cần huấn luyện lại
Ứng dụng tối ưu	Thử nghiệm, tác vụ thay đổi	Hệ thống sản xuất (Production), tác vụ lặp lại

FINE-TUNING VÀ FEW-SHOT LEARNING

Tiêu chí	Few-shot Learning	Fine-tuning
Cách hoạt động	Ví dụ trong prompt	Huấn luyện lại model
Tính linh hoạt	Cao	Thấp hơn
Chi phí runtime	Cao (prompt dài)	Thấp hơn
Khả năng mở rộng	Hạn chế	Tốt

FINE-TUNING MODELS WITH OPENAI

2. QUY TRÌNH FINE-TUNING



FINE-TUNING MODELS WITH OPENAI

3. THIẾT KẾ DỮ LIỆU HUẤN LUYỆN

PROMPT ENGINEERING VS. FINE-TUNING

```
{  
  "messages": [  
    {"role": "system", "content": "Bạn là một chuyên gia Refactoring mã nguồn Python."},  
    {"role": "user", "content": "Hãy tối ưu vòng lặp tính tổng bình phương này."},  
    {"role": "assistant", "content": "Bạn nên sử dụng np.dot(a, a) để đạt hiệu năng cao nhất.", "weight": 1}  
  ]  
}
```

You, 3 days ago • Uncommitted changes

Ý NGHĨA VÀ VAI TRÒ CỦA CÁC THÀNH PHẦN

- system (Hệ thống)
- user (Người dùng)
- assistant (Trợ lý)
- weight (Trọng số)

FINE-TUNING MODELS WITH OPENAI

4. CONTRASTIVE LEARNING TRONG FINE-TUNING

Contrastive Learning là kỹ thuật huấn luyện bằng cách cung cấp đồng thời cả ví dụ tích cực (hành vi mong muốn) và ví dụ tiêu cực (hành vi cần tránh).

LỢI ÍCH VƯỢT TRỘI

Tăng độ chính xác tuyệt đối (High Precision)

Giảm hành vi không mong muốn (Zero Noise)

Tối ưu hóa Pipeline tự động

FINE-TUNING MODELS WITH OPENAI

5. CASE STUDY: SINH CODE SẠCH (CLEAN CODE GENERATION)

VẤN ĐỀ TỒN ĐONG CỦA MÔ HÌNH MẶC ĐỊNH (BASE MODEL)

- Giải thích dư thừa (Verbosity)
- Lạm dụng Comment
- Cấu trúc chưa tinh gọn

GIẢI PHÁP THÔNG QUA FINE-TUNING CHUYÊN BIỆT

- Chế độ "Code-Only"
- Nguyên lý "No-Comment Policy"
- Module hóa tự động

KẾT QUẢ THỰC NGHIỆM

Chỉ số	Trước Fine-tuning	Sau Fine-tuning
Độ dài Output	Dài (bao gồm giải thích)	Ngắn gọn (chỉ có code)
Tính cấu trúc	Khối code lớn (Monolithic)	Đa hàm (Modularized)
Khả năng đọc	Trung bình (nhiều nhiễu)	Rất cao (Clean code)
Tự động hóa	Cần Parser để lọc code	Có thể thực thi trực tiếp

FINE-TUNING MODELS WITH OPENAI

6. CHI PHÍ FINE-TUNING

- Chi phí Huấn luyện (Training Costs)
- Chi phí Vận hành (Inference Costs)

BẢNG SO SÁNH TỔNG HỢP

Thành phần	Model Gốc (Prompt Engineering)	Model Fine-tuned
Giá đơn vị Token	Thấp (\$)	Cao (\$\$)
Độ dài System Prompt	Dài (Tốn token)	Rất ngắn (Tiết kiệm)
Độ dài Output	Trung bình/Dài	Ngắn nhất (Chỉ code)
Tổng chi phí/Request	Phụ thuộc vào độ dài chỉ dẫn	Thường tối ưu hơn 20-40%

FINE-TUNING MODELS WITH OPENAI

7. ĐÁNH GIÁ HIỆU QUẢ

SO SÁNH GIỮA MODEL GỐC VÀ MODEL FINE-TUNED

Tiêu chí	Model gốc	Fine-tuned
Độ dài output	Dài	Ngắn
Có giải thích	Có	Không
Structure	Trung bình	Tốt
Cost tổng	Cao	Tối ưu

**THANK YOU FOR
YOUR ATTENTION**

